

```
In [1]: import numpy as np
import pandas as pd

import IPython.display as dp
import matplotlib.pyplot as plt
import seaborn as sns

dp.set_matplotlib_formats("retina")
sns.set(style="whitegrid", font_scale=1.5)
sns.despine()

%matplotlib inline
```

```
/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_24492/321256244
3.py:8: DeprecationWarning: `set_matplotlib_formats` is deprecated since I
Python 7.23, directly use `matplotlib_inline.backend_inline.set_matplotlib
_formats()`
  dp.set_matplotlib_formats("retina")
<Figure size 640x480 with 0 Axes>
```

```
In [2]: from darts import TimeSeries
import darts.datasets as ds
import scipy.stats as sts
import statsmodels.api as sm
import statsmodels.tsa.api as tsa
from darts.utils.statistics import plot_acf
```

```
In [3]: air_pax = ds.AirPassengersDataset().load()
```

```
In [4]: train, val = air_pax.split_before(pd.Timestamp("19580101"))
```

# Lecture

## Forecasting

Given time series  $x_1, \dots, x_T, \dots$ , where  $y_t \in \mathbb{R}$  is measured at regular time intervals, we would like to **forecast** it, i.e. to find a function  $f_T$  such that:

$$x_{T+d} \approx \hat{x}_{T+d} = f_T(x_1, \dots, X_T, d)$$

where  $1 \leq d \leq D$  is the forecast lag and  $D$  is the forecast horizon.

## Simple methods of forecasting

- Mean

$$\hat{x}_{T+d} = \frac{1}{T} \sum_{t=1}^T x_t$$

- Moving average

$$\hat{x}_{T+d} = \frac{1}{k} \sum_{t=T-k}^T x_t$$

- Naïve

$$\hat{x}_{T+d} = x_T$$

- Naïve seasonal

$$\hat{x}_{T+d} = x_{T+d-ks}, \quad k = \lfloor (d-1)/s \rfloor + 1$$

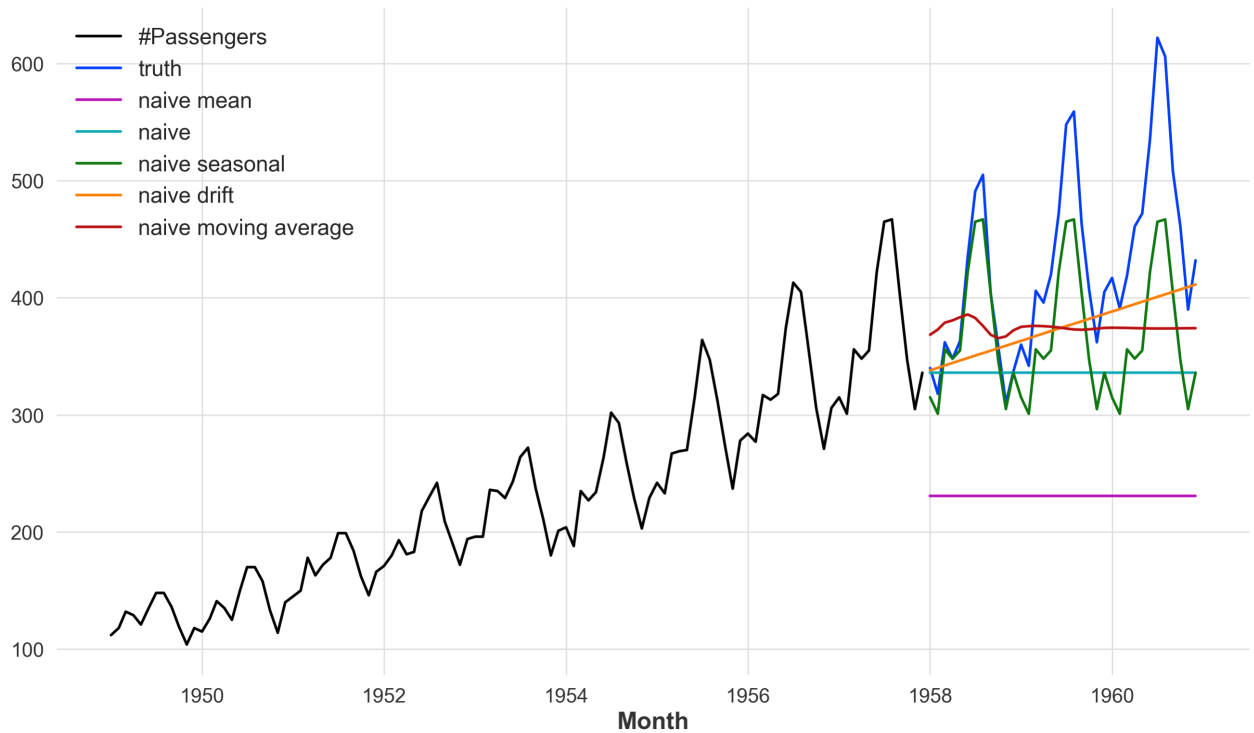
- Naïve drift (linear trend extrapolation)

$$\hat{x}_{T+d} = x_T + d \frac{x_T - x_1}{T - 1}$$

```
In [11]: from darts.models import NaiveMean, NaiveSeasonal, NaiveDrift, NaiveMovingAverage
```

```
In [12]: naive_mean_fcast = NaiveMean().fit(train).predict(len(val))
naive_fcast = NaiveSeasonal(K=1).fit(train).predict(len(val))
naive_seasonal_fcast = NaiveSeasonal(K=12).fit(train).predict(len(val))
naive_drift_fcast = NaiveDrift().fit(train).predict(len(val))
naive_ma_fcast = NaiveMovingAverage(12).fit(train).predict(len(val))

fig, ax = plt.subplots(figsize=(16,9))
train.plot(ax=ax);
val.plot(label="truth", ax=ax);
naive_mean_fcast.plot(label="naive mean", ax=ax);
naive_fcast.plot(label="naive", ax=ax);
naive_seasonal_fcast.plot(label="naive seasonal", ax=ax);
naive_drift_fcast.plot(label="naive drift", ax=ax);
naive_ma_fcast.plot(label="naive moving average", ax=ax);
```



## Smoothing

Smoothing is the name given to a general class of forecasting procedures that rely on a weighted sum of the past observations:

$$\hat{x}_{T+1} = c_0 x_T + c_1 x_{T-1} + \dots$$

Usually the weights are chosen such that they sum to one.

## Moving average is a smoothing

$$\hat{x}_{T+d} = \frac{1}{k} \sum_{t=T-k}^T x_t$$

Moving average is smoothing with constant coefficients within a window  $c_i = \frac{1}{k}$ .

## Exponential smoothing

For  $\alpha < 1$ :

$$\frac{1}{\alpha} = \sum_{i=1}^{\infty} (1 - \alpha)^i$$

So,

$$\sum_{i=1}^{\infty} \alpha(1 - \alpha)^i = 1$$

These form the coefficients for exponential smoothing:

$$\hat{x}_{T+1} = \alpha x_T + \alpha(1 - \alpha)x_{T-1} + \alpha(1 - \alpha)^2 x_{T-2} + \dots$$

## Simple exponential smoothing (Brown method)

$$\begin{aligned} \hat{x}_{t+1} &= l_t, \\ l_t &= \alpha x_t + (1 - \alpha)l_{t-1} \end{aligned}$$

- Suitable for series without trend or seasonality
- Depends on  $l_0$ :

$$\hat{x}_{t+1} = \sum_{j=1}^{T-1} \alpha(1 - \alpha)^j x_{T-j} + (1 - \alpha)^T l_0$$

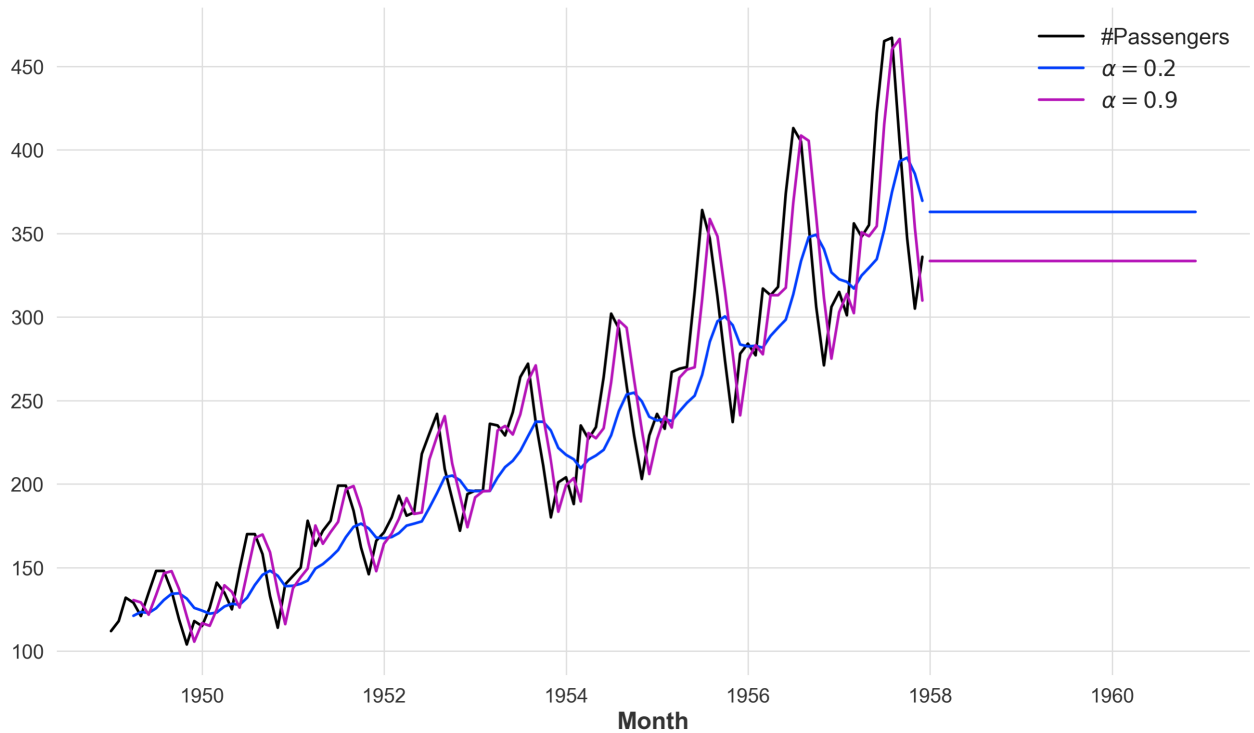
We can take  $l_0 = x_1$  or optimize for it

```
In [13]: from darts.models import ExponentialSmoothing
         from darts.utils.utils import ModelMode
```

```
In [21]: ses_02_bcast = ExponentialSmoothing(trend=ModelMode.NONE, seasonal=ModelMode.NONE)
         ses_09_bcast = ExponentialSmoothing(trend=ModelMode.NONE, seasonal=ModelMode.NONE)

         ses_02_fcast = ExponentialSmoothing(trend=ModelMode.NONE, seasonal=ModelMode.NONE)
         ses_09_fcast = ExponentialSmoothing(trend=ModelMode.NONE, seasonal=ModelMode.NONE)

         fig, ax = plt.subplots(figsize=(16,9))
         train.plot(ax=ax);
         ses_02_bcast.plot(label="$\\alpha = 0.2$", color="C1", ax=ax);
         ses_09_bcast.plot(label="$\\alpha = 0.9$", color="C2", ax=ax);
         ses_02_fcast.plot(label=None, color="C1", ax=ax);
         ses_09_fcast.plot(label=None, color="C2", ax=ax);
```



## Double exponential smoothing (Holt method)

Incorporates a term for linear additive trend:

$$\begin{aligned}\hat{x}_{t+d} &= l_t + db_t, \\ l_t &= \alpha x_t + (1 - \alpha)(l_{t-1} + b_{t-1}), \\ b_t &= \beta(l_t - l_{t-1}) + (1 - \beta)b_{t-1}.\end{aligned}$$

## Variants of DES: multiplicative trend

$$\begin{aligned}\hat{x}_{t+d} &= l_t b_t^d, \\ l_t &= \alpha x_t + (1 - \alpha)(l_{t-1} b_{t-1}), \\ b_t &= \beta \frac{l_t}{l_{t-1}} + (1 - \beta)b_{t-1}.\end{aligned}$$

## Variants of DES: damped trend

- Additive

$$\begin{aligned}\hat{x}_{t+d} &= l_t + (\phi + \phi^2 + \dots + \phi^d) b_t, \\ l_t &= \alpha x_t + (1 - \alpha)(l_{t-1} + \phi b_{t-1}), \\ b_t &= \beta(l_t - l_{t-1}) + (1 - \beta)\phi b_{t-1}.\end{aligned}$$

- Multiplicative

$$\hat{x}_{t+d} = l_t b_t^{\phi + \phi^2 + \dots + \phi^d},$$

$$l_t = \alpha x_t + (1 - \alpha) (l_{t-1} b_{t-1}^{\phi}),$$

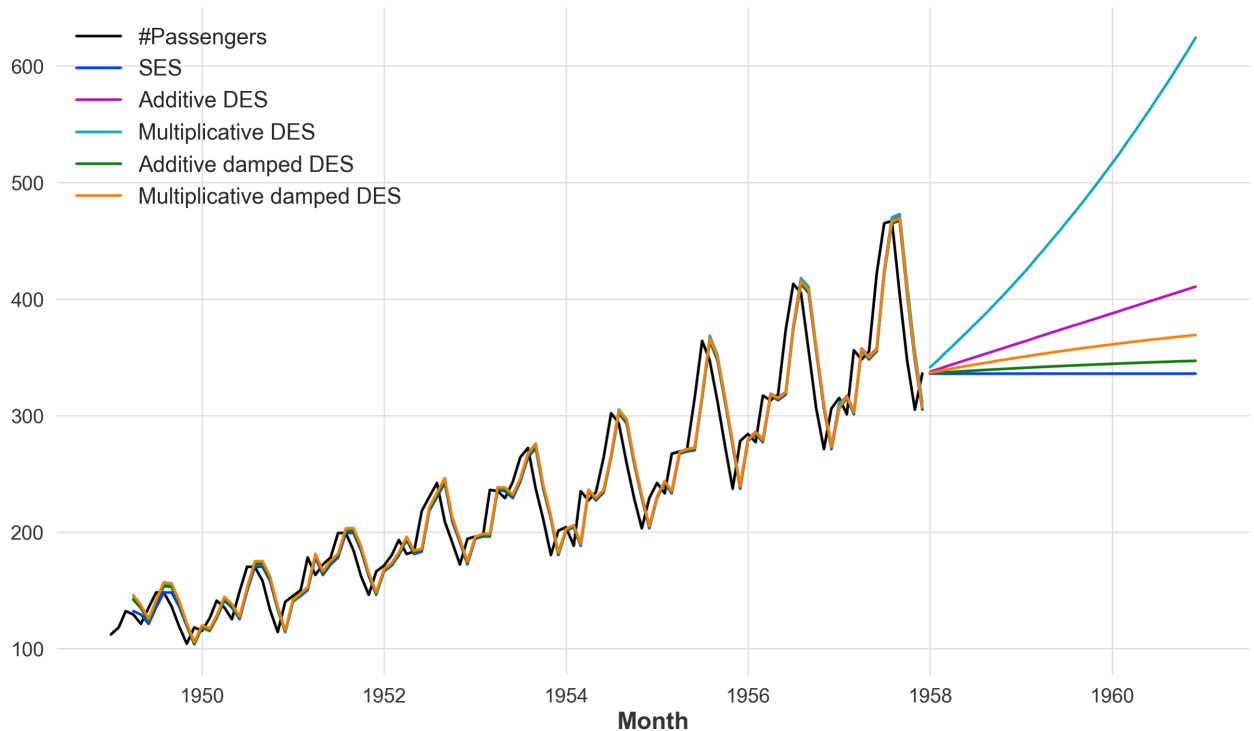
$$b_t = \beta \frac{l_t}{l_{t-1}} + (1 - \beta) b_{t-1}^{\phi}.$$

```
In [26]: ses = ExponentialSmoothing(trend=ModelMode.NONE, seasonal=ModelMode.NONE,
bes = ExponentialSmoothing(trend=ModelMode.ADDITIVE, seasonal=ModelMode.N
mbes = ExponentialSmoothing(trend=ModelMode.MULTIPLICATIVE, seasonal=Mode
dbes = ExponentialSmoothing(trend=ModelMode.ADDITIVE, seasonal=ModelMode.
dmbes = ExponentialSmoothing(trend=ModelMode.MULTIPLICATIVE, seasonal=Mod

ses_bcast = ses.historical_forecasts(train)
bes_bcast = bes.historical_forecasts(train)
mbes_bcast = mbes.historical_forecasts(train)
dbes_bcast = dbes.historical_forecasts(train)
dmbes_bcast = dmbes.historical_forecasts(train)

ses_fcast = ses.fit(train).predict(len(val))
bes_fcast = bes.fit(train).predict(len(val))
mbes_fcast = mbes.fit(train).predict(len(val))
dbes_fcast = dbes.fit(train).predict(len(val))
dmbes_fcast = dmbes.fit(train).predict(len(val))

fig, ax = plt.subplots(figsize=(16,9))
train.plot(ax=ax);
ses_bcast.plot(label="SES", color="C1", ax=ax);
bes_bcast.plot(label="Additive DES", color="C2", ax=ax);
mbes_bcast.plot(label="Multiplicative DES", color="C3", ax=ax);
dbes_bcast.plot(label="Additive damped DES", color="C4", ax=ax);
dmbes_bcast.plot(label="Multiplicative damped DES", color="C5", ax=ax);
ses_fcast.plot(label=None, color="C1", ax=ax);
bes_fcast.plot(label=None, color="C2", ax=ax);
mbes_fcast.plot(label=None, color="C3", ax=ax);
dbes_fcast.plot(label=None, color="C4", ax=ax);
dmbes_fcast.plot(label=None, color="C5", ax=ax);
```



## Triple exponential smoothing (Holt-Winters method)

Incorporates a term for linear additive seasonality of cycle length  $m$ :

$$\begin{aligned}\hat{x}_{t+d} &= l_t + db_t + s_{t-m+(d \bmod m)}, \\ l_t &= \alpha (x_t - s_{t-m}) + (1 - \alpha) (l_{t-1} + b_{t-1}), \\ b_t &= \beta (l_t - l_{t-1}) + (1 - \beta) b_{t-1}, \\ s_t &= \gamma (x_t - l_{t-1} - b_{t-1}) + (1 - \gamma) s_{t-m}.\end{aligned}$$

Multiplicative seasonality is also possible:

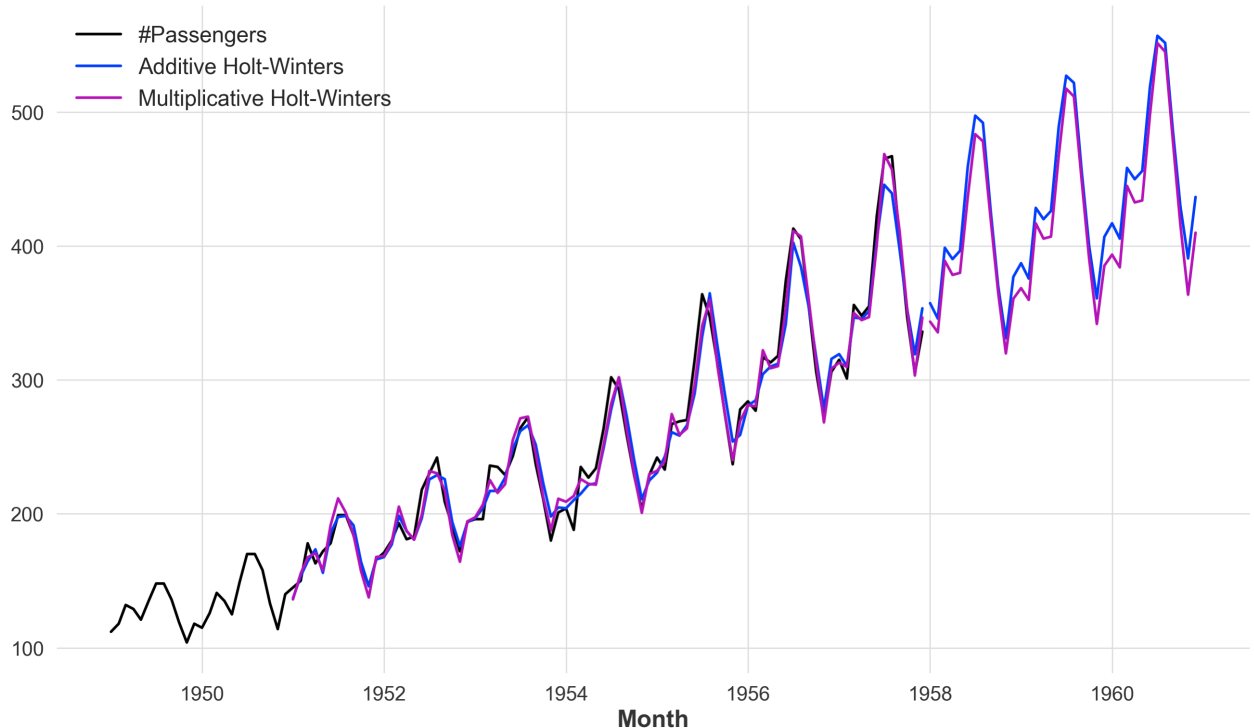
$$\begin{aligned}\hat{x}_{t+d|t} &= (l_t + db_t) s_{t-m+(d \bmod m)}, \\ l_t &= \alpha \frac{x_t}{s_{t-m}} + (1 - \alpha) (l_{t-1} + b_{t-1}), \\ b_t &= \beta (l_t - l_{t-1}) + (1 - \beta) b_{t-1}, \\ s_t &= \gamma \frac{x_t}{l_{t-1} + b_{t-1}} + (1 - \gamma) s_{t-m}.\end{aligned}$$

```
In [28]: ets_additive = ExponentialSmoothing(trend=ModelMode.ADDITIVE, seasonal=ModelMode.ADDITIVE)
ets_mult = ExponentialSmoothing(trend=ModelMode.ADDITIVE, seasonal=ModelMode.MULTIPLICATIVE)

ets_additive_bcast = ets_additive.historical_forecasts(train)
ets_mult_bcast = ets_mult.historical_forecasts(train)

ets_additive_fcast = ets_additive.fit(train).predict(len(val))
ets_mult_fcast = ets_mult.fit(train).predict(len(val))
```

```
fig, ax = plt.subplots(figsize=(16,9))
train.plot(ax=ax);
ets_additive_bcast.plot(label="Additive Holt-Winters", color="C1", ax=ax)
ets_mult_bcast.plot(label="Multiplicative Holt-Winters", color="C2", ax=ax)
ets_additive_fcast.plot(label=None, color="C1", ax=ax);
ets_mult_fcast.plot(label=None, color="C2", ax=ax);
```



## ETS models overlook

|                       | Trend   | Seasonality |                            |
|-----------------------|---------|-------------|----------------------------|
|                       |         | None        | Additive    Multiplicative |
| None                  | (N, N)  | (N, A)      | (N, M)                     |
| Additive              | (A, N)  | (A, A)      | (A, M)                     |
| Additive damped       | (Ad, N) | (Ad, A)     | (Ad, M)                    |
| Multiplicative        | (M, N)  | (M, A)      | (M, M)                     |
| Multiplicative damped | (Md, N) | (Md, A)     | (Md, M)                    |

Also we can model the error as additive or multiplicative for probabilistic forecast, so we get  $ETS(\cdot, \cdot, \cdot)$ .

## Other models

There is a model named Theta method, which is a specific variant of exponential

smoothing:

$$\hat{x}_{t+h} = \frac{\theta - 1}{\theta} \hat{b}_0 \left[ h - 1 + \frac{1}{\hat{\alpha}} - \frac{(1 - \hat{\alpha})^t}{\hat{\alpha}} \right] + l_{t+h}$$

where  $b_0$  is estimated from OLS:

$$x_t = a_0 + b_0(t - 1) + \varepsilon_t$$

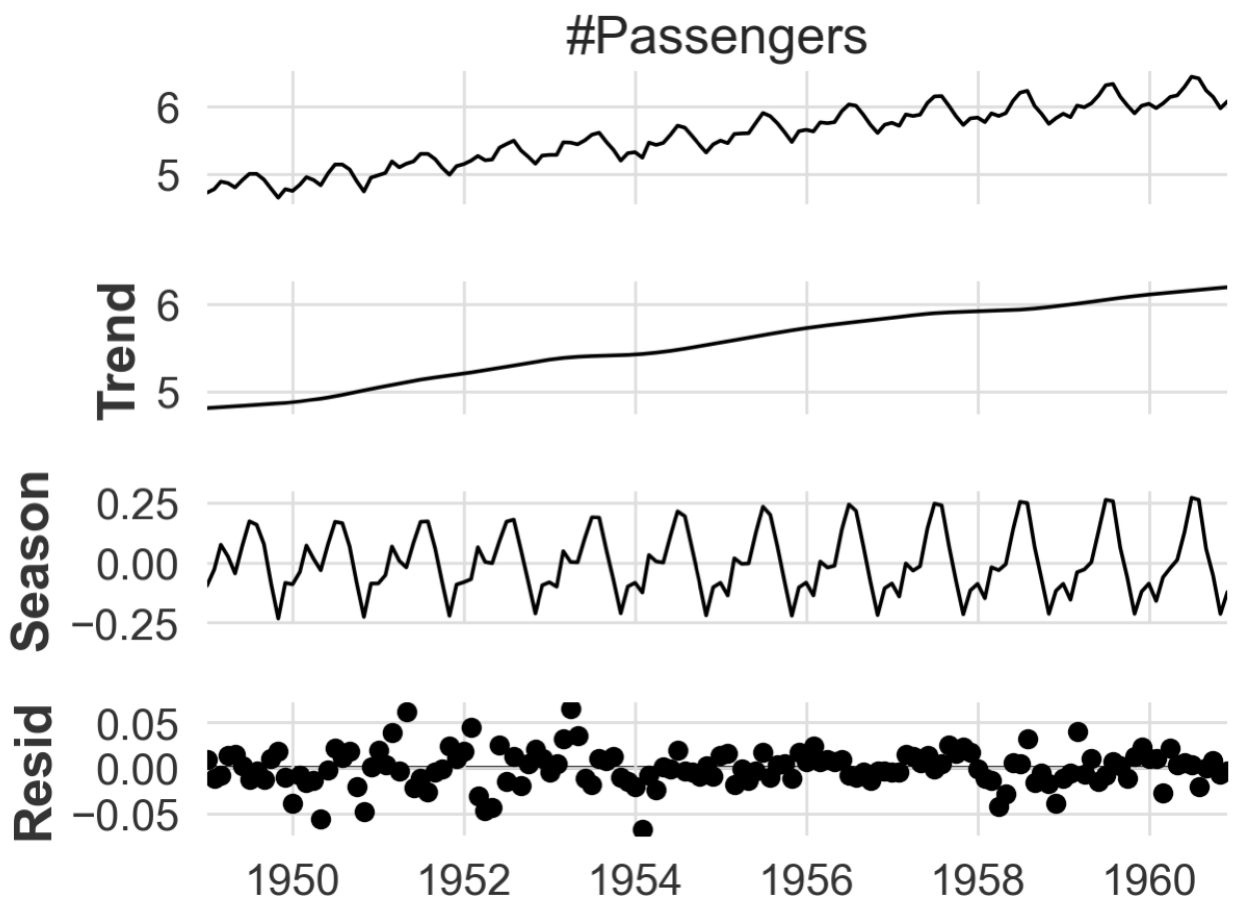
and  $\alpha$  is estimated from SES:

$$l_t = (1 - \alpha)x_t + \alpha l_{t-1}$$

## Webinar

### STL decomposition

```
In [49]: tsa.STL(air_pax.map(lambda ts, x: np.log(x)).pd_dataframe().iloc[:, 0]).f
```



```
In [50]: model = tsa.STLForecast(train.map(lambda ts, x: np.log(x)).pd_dataframe())
         fitted_model = model.fit()
         fcast = TimeSeries.from_dataframe(fitted_model.forecast(len(val)).to_frame)
```

```

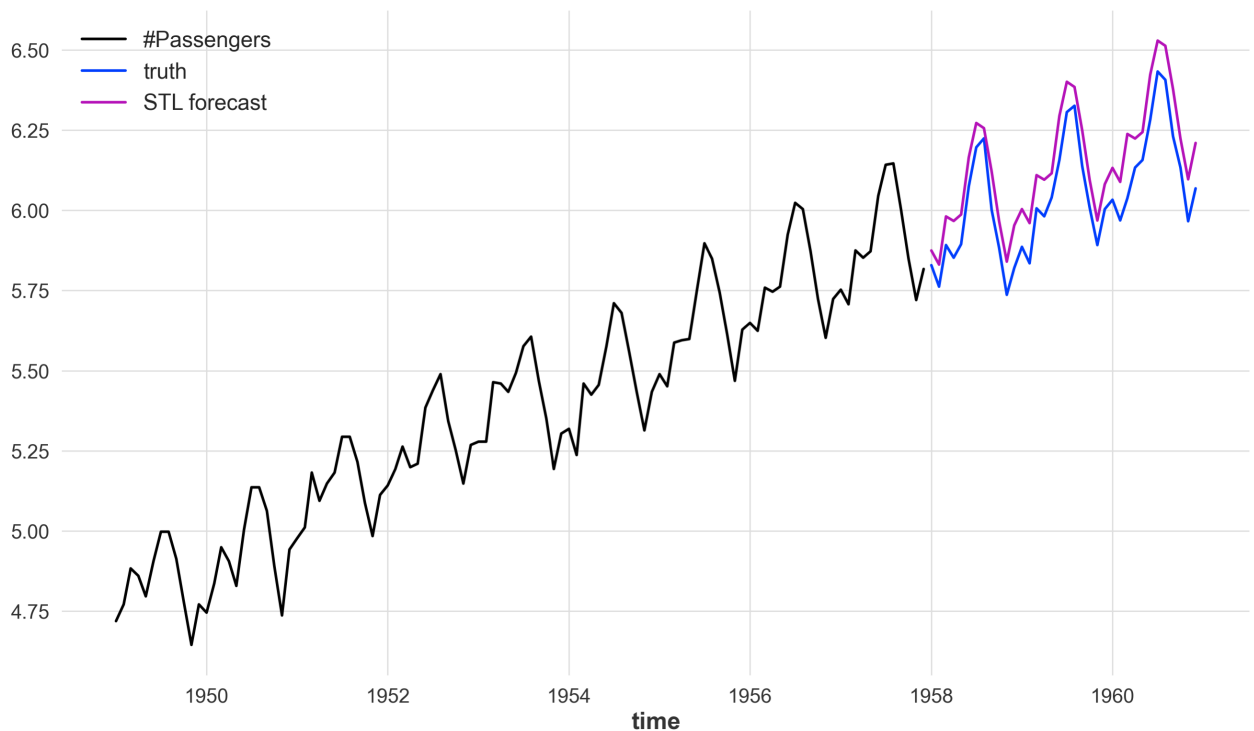
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/tsa/statespace/sarimax.py:966: UserWarning: Non-stationary starting autoregressive parameters found. Using zeros as starting parameters.
  warn('Non-stationary starting autoregressive parameters')
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/tsa/statespace/sarimax.py:978: UserWarning: Non-invertible starting MA parameters found. Using zeros as starting parameters.
  warn('Non-invertible starting MA parameters found.')
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
  warnings.warn("Maximum Likelihood optimization failed to ")

```

```

In [51]: fig, ax = plt.subplots(figsize=(16,9))
        train.map(lambda ts, x: np.log(x)).plot(ax=ax);
        val.map(lambda ts, x: np.log(x)).plot(label="truth", ax=ax);
        fcast.plot(label="STL forecast", ax=ax);

```



## Quality measures

- MSE

$$MSE = \frac{1}{T - R + 1} \sum_{t=R}^T (\hat{x}_t - x_t)^2$$

- MAE

$$MAE = \frac{1}{T - R + 1} \sum_{t=R}^T |\hat{x}_t - x_t|.$$

- MAPE

$$MAPE = \frac{100}{T - R + 1} \sum_{t=R}^T \left| \frac{\hat{x}_t - x_t}{x_t} \right|.$$

- SMAPE

$$SMAPE = \frac{200}{T - R + 1} \sum_{t=R}^T \left| \frac{\hat{x}_t - x_t}{\hat{x}_t + x_t} \right|.$$

- MASE

$$MASE = \frac{1}{T - R + 1} \sum_{t=R}^T |\hat{x}_t - x_t| / \frac{1}{T - 1} \sum_{t=2}^T |x_t - x_{t-1}|.$$

## AutoETS

```
In [52]: from darts.models import StatsForecastAutoETS
```

```
In [53]: auto_ets = StatsForecastAutoETS(season_length=12)
auto_ets.fit(train)
```

```
Out[53]: StatsForecastAutoETS(add_encoders=None, season_length=12)
```

```
In [56]: auto_fcast = auto_ets.predict(len(val))
```

```
In [57]: auto_ets.model.model_["method"]
```

```
Out[57]: 'ETS(M,N,M)'
```

```
In [59]: fig, ax = plt.subplots(figsize=(16,9))
train.plot(ax=ax);
val.plot(label="truth", ax=ax);
auto_fcast.plot(label="AutoETS forecast", ax=ax);
```

